



DIGITAL HEALTH FINANCING HACKATHON 2026 · CLAIMS MANAGEMENT & FRAUD DETECTION

Fraud Anomaly Detection

An explainable, two-layer fraud detection module built natively into openIMIS — trained and validated on 422,382 real Kenyan health insurance claims.

LOW

MEDIUM

HIGH

Every claim, scored automatically, with a reason a human can read.

Jenipher Mawia · Lewis Munyi

KeHIA × @iLabAfrica, Strathmore University



DIGITAL HEALTH FINANCING HACKATHON 2026 · CLAIMS MANAGEMENT & FRAUD DETECTION

Team Immara:

1. Jenipher Mawia
2. Lewis Munyi
3. Fredrick Benard

THE PROBLEM

Claims adjudication is manual - it is slow and it misses fraud



Every claim is a judgement call

A human reviewer reads each claim and decides. With thousands arriving daily, queues stretch to days or weeks.



Fraud hides across claims

A reviewer **sees one claim at a time** — not the provider that has quietly overbilled for months.



No automated safety net

openIMIS — serving 38.8M beneficiaries in 14 countries — currently has no fraud detection layer.

The cost is real: in the dataset behind this project, **KES 565 million** — 12.9% of everything invoiced — was disputed between what providers billed and what the insurer paid.

GROUNDING IN REAL DATA

Seven years of anonymised claims from a Kenyan medical insurer

422,382

claims

KES 4.38B

invoiced

KES 565M

invoice-settlement variance (12.9%)

1,019

healthcare providers

103,069

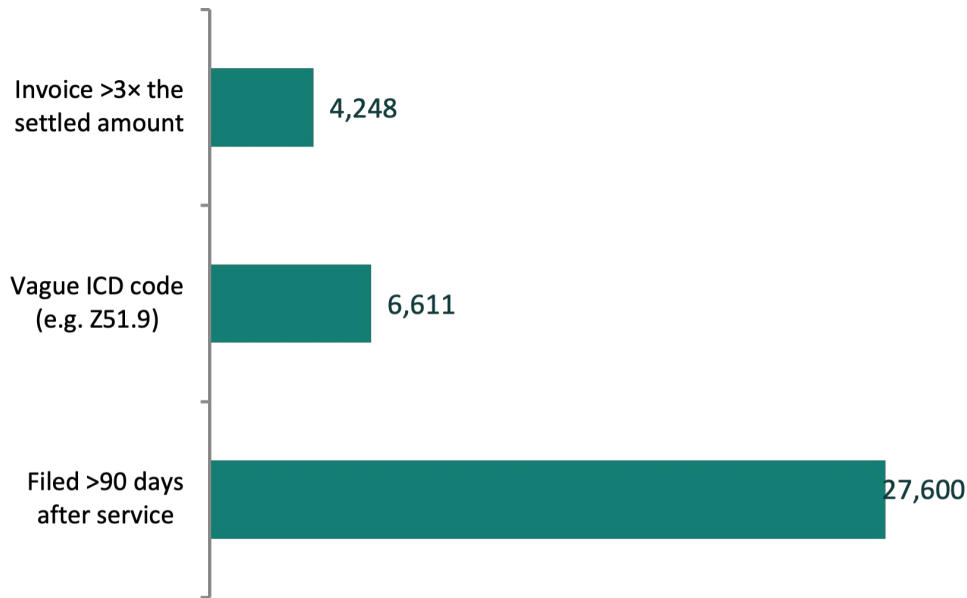
unique members

2018 – 2025

date range

*All member and claim identifiers were **SHA-256 hashed before analysis**. No raw PII ever entered the repository.*

Fraud patterns were hiding in plain sight



27,600 claims filed months late

Some 6–8 months after the service date — a strong backdating signal.



Extreme outliers exist

Largest single invoice: KES 11.6M, against a median claim of KES 3,949.



Vague codes disguise big bills

6,611 claims used catch-all ICD codes like Z51.9 — “medical care, unspecified.”



10.4% of claims look suspicious

Settled below 80% of invoice — the insurer itself already disputed them.

These four findings became the blueprint for the rules engine and the model's features.

Two detection layers, one human decision



Layer 1 — Rules engine

Five deterministic rules derived from the patterns in the data. Instant, free to run and every flag comes with a plain-language reason.

Explainable



Layer 2 — Isolation Forest

An unsupervised ML model trained on all 422k claims. Catches the subtle, evolving schemes no one thought to write a rule for.

Adaptive



Human in the loop

Reviewers see a risk badge and the reasons, then approve, reject, or escalate. Their overrides feed the next retraining cycle.

Accountable

Every claim gets a badge:

LOW

MEDIUM

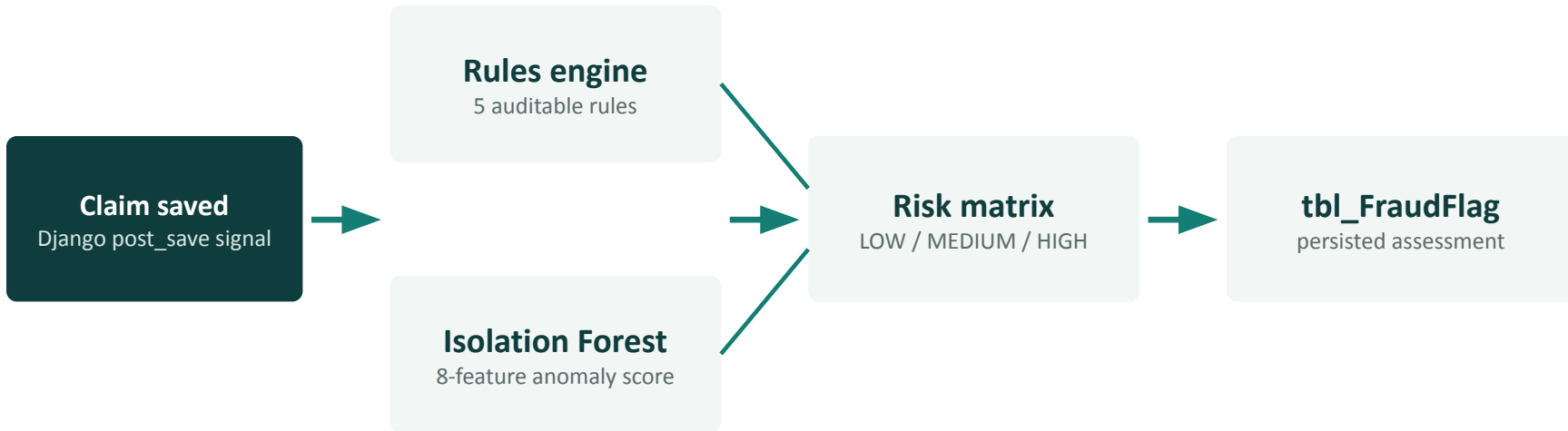
HIGH

High-risk claims jump the review queue; low-risk claims get paid faster.

HOW IT WORKS

Scoring runs automatically inside openIMIS

Decision matrix: 2+ rules → HIGH · rules + ML anomaly → HIGH · one of the two → MEDIUM · neither → LOW



Surfaced through every interface openIMIS speaks:



Five rules a reviewer can read out loud

1 **Claim lag exceeds 90 days**

Filed long after the service — the classic backdating signal (27,600 claims matched).

2 **Invoice inflation above 3×**

Billed more than triple what was approved — deliberate overbilling (4,248 claims).

3 **Vague ICD code used**

Catch-all codes like Z51.9 give no clinical justification (6,611 claims).

4 **Claim filed after its audit date**

Logically impossible — points to record tampering.

5 **High-value claim, vague diagnosis**

Over KES 20,000 without a precise code — big bills need real justification.

Thresholds are plain constants an administrator can tune — no ML expertise required.

Isolation Forest — learning what “normal” looks like

Why unsupervised? No labelled fraud data exists — no insurer hands out a list of confirmed fraudsters. Isolation Forest learns the shape of normal claims from all 422k rows and scores how far each new claim strays from it.

More negative score = more anomalous. The score is always shown next to the fired rules, never alone — reviewers see the “why,” not just a number.

Feature	Fraud signal
Invoice ÷ settled ratio	Overbilling
Days from service to claim	Backdating
Vague ICD flag	Diagnosis disguise
Provider average inflation	Provider-level pattern
Provider claim volume	Volume anomaly
Member claim count	Ghost beneficiary
Amount vs benefit benchmark	Benchmark deviation

Why Isolation Forest?

- **No ground-truth labels (Hard constraint)** *proxy_fraud_label* ("settled < 80% of invoice") is only used for evaluation — it's never fed to the model during training. *model.fit(X_scaled)* in *retrain_fraud_model.py* takes X only, no y.
- **Fraud is rare and unknown-shaped.** We are looking for the "weird" claims, not claims that match a known template.
- **Few Hyperparameters.** 422k rows, 8 numeric features. Tabular, moderate size.
- **Fast Inference** — $O(\log n)$ inference is cheap inside a *post_save* signal

Unsupervised learning alternatives

Model	vs. Isolation Forest
Local Outlier Factor (LOF)	Good at local anomalies IF misses. But it's memory- and compute-heavy (needs neighbour lookups) and doesn't cleanly score new points out-of-the-box (novelty=True needed). Poor fit for 422k rows scored live.
One-Class SVM	Powerful but scales badly (roughly $O(n^2)$); very slow to train at our data size and sensitive to kernel/params. IsoForest is far more practical.
Elliptic Envelope (robust covariance)	Assumes data is roughly one Gaussian blob — claims data is skewed and multi-modal , so this underperforms.
Autoencoder (neural net)	Can capture complex patterns and often beats Isolation Forest on rich data — but needs more data-engineering, GPU/time, tuning and is far less explainable. Overkill for 8 tabular features
DBSCAN / clustering	Not designed for scoring new points ; hard to tune eps; not incremental
Gaussian Mixture Models	Reasonable, but assumes a mixture-of-Gaussians shape and is fiddlier than IsoForest with little upside here.

Supervised alternatives

Model	Pros	Cons
Gradient-boosted trees (XGBoost / LightGBM)	Usually best-in-class on tabular fraud; handles imbalance; feature importances	Needs labels ; needs care with class imbalance
Random Forest (supervised)	Robust, explainable, strong baseline	Needs labels
Logistic Regression	Simple, transparent, great baseline & regulatory-friendly	Lower ceiling; needs labels

Supervised models usually outperform any anomaly detector, because they learn the actual decision boundary instead of just "unusualness" - But they need to have **labels** of actual real data

DOES IT WORK?

Validated on 84,477 held-out claims — reported honestly

88%

overall accuracy

0.8474

ROC-AUC — well above
chance

71,523

normal claims correctly
cleared (94%)

2,751

suspicious claims caught

	Predicted normal	Predicted suspicious
Actually normal	71,523	4,072
Actually suspicious	6,131	2,751

Caveat: with no true fraud labels, we **evaluated against a proxy** — claims the insurer settled below 80% of invoice. Not every such claim is fraud, which depresses measured precision (0.30 on the suspicious class). **The rules layer compensates: two or more fired rules reach HIGH with no ML at all.**

The model flags — a person decides



Human review only

The system never auto-rejects a claim. Flags prioritise the review queue; **reviewers make every final call.**



Feedback loop built in

Every APPROVE / REJECT / ESCALATE override is stored and surfaced at the next **retraining** — **the model learns from its corrections.**



Privacy first

All **persons' identifiers SHA-256 hashed before training.** Raw PII never entered the repository.



Limits stated openly

Trained on one insurer's data — deploying elsewhere means retraining locally. Novel schemes need a retrain to be caught.

Everything runs on the real stack — nothing mocked

1

Triage dashboard

Every scored claim with its risk badge, can be filtered by level — the reviewer's morning view.

`GET /flags/`

2

Score a claim, live

Enter a claim, watch the rules fire and the anomaly score appear, with the category explained in one sentence.

`POST /score/`

3

Reviewer override

Disagree with the model, record a decision with notes — and it's stored for the next retraining cycle.

`POST /override/`

Real-world data and infrastructure fought back



No fraud labels exist

Chose unsupervised learning and built an honest proxy label (settled < 80% of invoice) for evaluation — and said so openly in the performance report.



158,539 corrupted audit dates

A whole date column arrived as Excel serial numbers. We detected the pattern, converted them and kept a key fraud feature alive.



“Duplicates” that weren't

86k repeated claim numbers turned out to be line items, not resubmissions — a discovery that redesigned our duplicate-detection strategy.



Upstream test-database bug

The openIMIS test runner can't build its test DB from scratch. We engineered a template-clone workaround — 36 tests now pass reliably.

Each one is documented in the repository.

Status Report:

✓	Scoring integrated via Django signal	Runs automatically on every claim save
✓	REST + GraphQL APIs	5 endpoints, paginated, validated
✓	FHIR R4 ClaimResponse extension	fhir_extensions.py
✓	Performance report	Precision, recall, F1, confusion matrix
✓	Docker-based openIMIS deployment	Compose overlay on openimis-dist_dkr
✓	Draft PR to openIMIS on GitHub	Open for community review
✓	React risk badge in claims list	In progress — shown in Streamlit today



Challenges:

- Data collection & data privacy issues
- Data from different insurers comes in different formats - need to reconcile them before data analysis
- No fraud labels exist in the data
- Had to use Isolation Forest rather than supervised learning models that would ideally perform better.



LOW

MEDIUM

HIGH

Faster payment for honest claims. Harder times for dishonest ones.

What's next

- Ship the React risk badge into the openIMIS claims list
- Scheduled retraining with reviewer overrides folded in
- Provider-level aggregate features computed in real time
- Include an ETL data engineering layer that ingests the data and transforms it cleanly before any analysis

Open source, documented and reproducible:

github.com/Jenn-mawia/openimis-be-fraud-detect_py

Asante sana — questions welcome